

基于 claude 进行 webUI 自动化

一、安装 claude 和 mcp

1、安装 mcp

在 cmd 中输入安装命名 `npm install -g @playwright/mcp`
等待完成，查看 `npm list` 是否出现 playwright

```
C:\Users\admin>npm list
admin@ C:\Users\admin
+-- @anthropic-ai/claude-code@2.1.79
+-- @playwright/test@1.58.2
`-- g@2.0.1
```

2、将 mcp_playwright 集成到 claude

打开 claude，告诉 claude，本机已存在 mcp_playwright，将 mcp 与 claude 进行集成，剩下的交给 claude，完成后，可以在 claude 中查看

```
> 你是否有mcp_playwright能力

● 有。
  我这里可用的是 mcp__playwright__browser_* 这一组浏览器工具，能做这些事：

  - 打开/关闭页面
  - 访问网址
  - 点击、输入、选择下拉框
  - 截图
  - 抓取页面快照
  - 读取控制台日志
  - 读取网络请求
  - 执行页面内 JS / Playwright 代码
  - 多标签页操作
  - 文件上传、键盘操作、等待元素/文本出现

  如果你要，我可以直接帮你：

  - 打开某个网页并检查问题
  - 自动化一段浏览器操作流程
  - 抓接口请求/报错
```

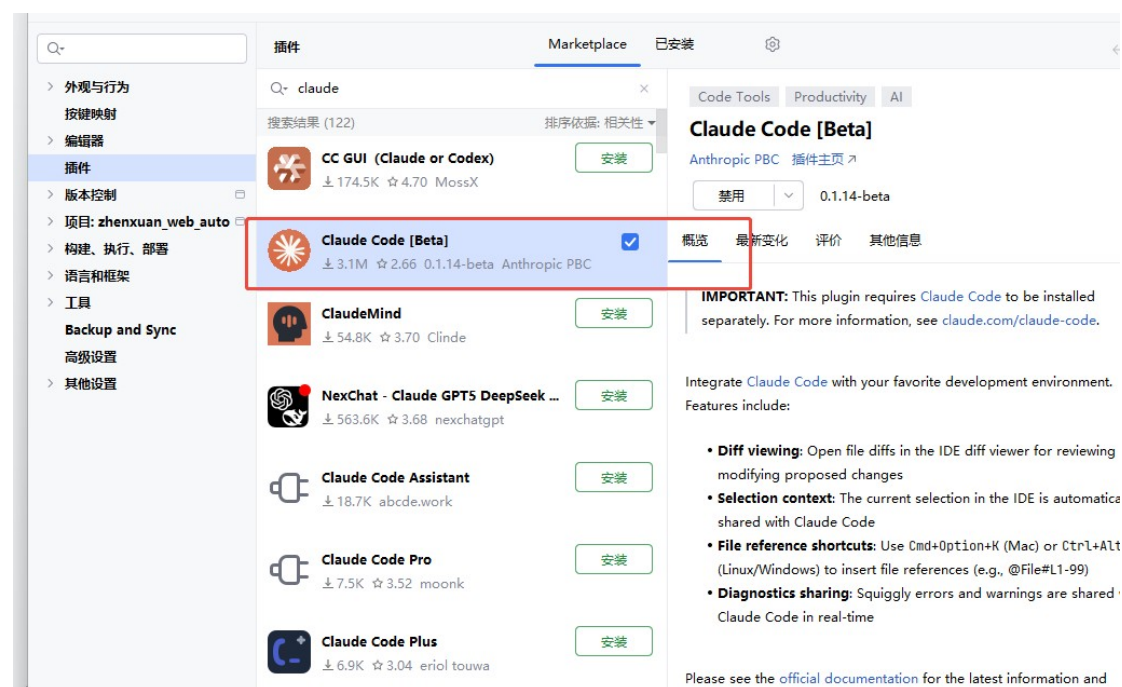
也可以在 settings.json 中查看

```
{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "sk-ant-api01-...",
    "ANTHROPIC_BASE_URL": "http://localhost:8082",
    "CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC": "1",
    "ANTHROPIC_CUSTOM_MODEL_OPTION": "gpt-5.4",
    "ANTHROPIC_CUSTOM_MODEL_OPTION_NAME": "gpt-5.4"
  },
  "permissions": {
    "allow": [],
    "deny": []
  },
  "model": "gpt-5.4",
  "enabledMcpjsonServers": [
    "playwright"
  ]
}
```

二、在项目中集成 claude 和 mcp

1、在 pycharm 安装 claudecode 插件

打开设置->插件->marketplace->输出 claude 进行安装



2、在项目中集成

在项目中打开 **claude** 插件，输入对话，将 **mcp_playwright** 集成到项目中，剩下的交给 **ai** 就可以了

Context

用户希望在当前 Python UI 自动化项目中集成 **mcp_playwright**。现有项目已经使用 Playwright 与 **pytest-playwright** 作为测试执行基础，因此这次集成的重点不！客户端可以直接连接 Playwright 能力，用于页面探查、调试、录制和辅助定位问题，同时尽量不影响当前 **pytest** 执行链路。

Recommended approach

1. 先区分两层集成：

- Claude Code / harness 层：把 **mcp_playwright** 注册为 MCP server，供当前开发会话使用。

- 项目仓库层：补充项目依赖、启动说明，以及一个最小可用的接入脚本/配置，让团队成员在仓库内可复用该能力。

2. 复用现有 Playwright 基础，不改动主测试模型：

- 保持 **pytest-playwright** fixture 体系不变，核心沿用 **conftest.py** 与 **test_case/conftest.py** 中的浏览器、context、page 管理。

- 不把 **mcp_playwright** 硬塞进 **run.py** / **base/tool/process_test.py** 的常规回归执行流程，避免影响 Jenkins/Allure/企业微信通知链路。

- 把 MCP 能力定位为“本地调试与辅助操作入口”，而不是批量测试执行入口。

3. 优先改动的关键文件：

- **requirements.txt**

- 视 **mcp_playwright** 的实际安装方式补充依赖（若它通过 Node 包提供，则这里不加 Python 依赖，而是在项目说明中声明前置条件）。

- **conftest.py**

- 如需统一浏览器启动参数，可把 MCP 调试所需的浏览器参数、trace/video/screenshot 相关配置抽成可复用配置位，但不改变现有 fixture 对外接口。

- **test_case/conftest.py**

- 如需让 MCP 调试复用现有登录态，可在这里梳理并暴露更稳定的 **storage_state** 生成/读取路径约定。

- **base/spec/spec_playwright.py**

- 这里已有 **sync_playwright()** 直连封装，适合整理成“独立调试入口”的复用点；若需要仓库内提供 MCP 调试脚本，这里是优先复用的位置。

三、让 claude 熟悉项目

1、熟悉项目框架

通过对话，让 **claude code** 学习项目框架层级和命令方式（读取项目框架层级，进行学习，理解框架层级和文件与项目的命名规则）

可以看到，**claude** 已经完全读取项目，并且已经学习会了整个项目

终端 Claude Code x + -

5. 最后回归检查 Allure 结果生成链路不受影响，重点关注：

- D:\auto\zhenxuan_web_auto\run.py

- D:\auto\zhenxuan_web_auto\base\tool\process_test.py

- D:\auto\zhenxuan_web_auto\conftest.py

读取项目框架层级，进行学习，理解框架层级和文件与项目的命名规则

● Explore(Explore framework layering) Sonnet 4.6

L Done (0 tool uses · 0 tokens · 2s)

(ctrl+o to expand)

Searching for 4 patterns, reading 6 files... (ctrl+o to expand)

L test_case\test_甄迭后台\test_管理员\test_基础设置\test_基础设置_虚拟订单列表.py

* 正在梳理项目框架层级... (49s · ↓ 470 tokens)

L ■ 梳理项目框架层级

plan mode on (meta+m to cycle) · esc to interrupt · ctrl+t to hide tasks

```
- tester_admin_virtual_order_class_list
```

9. 这个框架的实际设计思想

可以概括成：

pytest 用例层

- 只负责声明测试和调用 fixture

case 层

- 面向测试步骤，适合截图、断言、报告描述

feature 层

- 面向业务流程组合

meta 层

- 面向页面对象和元素操作

base 层

- 提供框架底座能力

这是一种 POM + 业务分层 + **pytest fixture** 驱动的自动化测试框架。

10. 当前项目里能看出的命名特点

这个项目命名规则总体清晰，但存在一些历史遗留的不统一：

1

下面是这个项目的调用关系图，按你当前打开的“虚拟订单列表”链路来画，最能代表整个框架。

整体调用链

[run.py](#)

↓

[base/tool/process_test.py](#)

↓

pytest 收集 test_case/**/*.py

↓

全局 [confptest.py](#) 初始化浏览器 / context / page / 环境

↓

模块 [confptest.py](#) 构造 tester_xxx fixture

↓

test_xxx.py 调用 tester 对象的方法

↓

business/case_objects

↓

business/feature_objects

↓

business/meta_objects

↓

Playwright Page 实际操作页面

具体到“虚拟订单列表”这条链路

[run.py:21](#)

run_test(mark, env)

↓

[base/tool/process_test.py:74](#)

ProcessTesting.run(*args)



